



Point Cloud Acceleration by Exploiting Geometric Similarity

Cen Chen

chencen@scut.edu.cn

School of Future Technology, South
China University of Technology &
Pazhou Lab
Guangzhou, China

Xiaofeng Zou*

zouxiaofeng@hnu.edu.cn

School of Future Technology, South
China University of Technology
Guangzhou, China

Hongen Shao

ftshaohongen@mail.scut.edu.cn

School of Future Technology, South
China University of Technology
Guangzhou, China

Yangfan Li

liyafan37@csu.edu.cn

School of Computer Science and
Engineering, Central South University
Changsha, China

Kenli Li

lkl@hnu.edu.cn

College of Information Science and
Engineering, Hunan University
Changsha, China

ABSTRACT

Deep learning on point clouds has attracted increasing attention for various emerging 3D computer vision applications, such as autonomous driving, robotics, and virtual reality. These applications interact with people in real-time on edge devices and thus require low latency and low energy. To accelerate the execution of deep neural networks (DNNs) on point clouds, some customized accelerators have been proposed, which achieved a significantly higher performance with reduced energy consumption than GPUs and existing DNN accelerators.

In this work, we reveal that DNNs execution on geometrically adjacent points exhibits similar values and relations, and exhibits a large amount of redundant computation and communication due to the correlations. To address this issue, we propose GDPCA, a geometry-aware differential point cloud accelerator, which can exploit geometric similarity to reduce these redundancies for point cloud neural networks. GDPCA is supported by an algorithm and architecture co-design. Our proposed algorithm can discover and reduce computation and communication redundancies with geometry-aware and differential execution mechanisms. Then a novel architecture is designed to support the proposed algorithm and transform the redundancy reduction into performance improvement. GDPCA performs the same computations and gives the same accuracy as traditional point cloud neural networks. To the best of our knowledge, GDPCA is the first accelerator that can reduce execution redundancies for point cloud neural networks by exploiting geometric similarity. Our proposed GDPCA system gains an average of $2.9\times$ speedup and $2.7\times$ energy efficiency over state-of-the-art accelerators for point cloud neural networks.

*Corresponding Author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MICRO '23, October 28–November 01, 2023, Toronto, ON, Canada

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0329-4/23/10...\$15.00

<https://doi.org/10.1145/3613424.3614290>

CCS CONCEPTS

• **Computer systems organization** → Neural networks; • **Hardware** → *Hardware accelerators*.

KEYWORDS

Hardware Accelerator, Point cloud, Redundancy-Aware Computation, Software-Hardware Co-Design

ACM Reference Format:

Cen Chen, Xiaofeng Zou, Hongen Shao, Yangfan Li, and Kenli Li. 2023. Point Cloud Acceleration by Exploiting Geometric Similarity. In *56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '23)*, October 28–November 01, 2023, Toronto, ON, Canada. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3613424.3614290>

1 INTRODUCTION

With the rapid development of 3D acquisition technologies, 3D sensors (e.g., LiDARs, and RGB-D cameras) are becoming increasingly available and affordable. 3D vision data generated by these sensors can provide rich geometric, shape and scale information of objects [13, 14], rather than 2D projections of objects (i.e., images). Compared with 2D images, 3D vision data provides an opportunity for a better understanding of the surrounding environment [5, 6]. The use of 3D vision technology provides a highly realistic simulation of real-world scenes, making it a significant and widely applicable tool in various fields [15]. Recently, deep learning on point clouds has attracted increasing attention and has been applied in various applications such as autonomous driving [5, 6], robotics [29], virtual reality (VR) [36], and augmented reality (AR) [40]. These applications require real-time interactions with humans, and thus it is crucial to emphasize not only high accuracy, but also low latency and energy consumption [5, 15].

Due to the unstructured and unordered nature of 3D point clouds, early deep learning based works usually convert raw point clouds into 2D images [2, 18, 35, 44], or 3D voxels [38, 50]. However, these methods introduce a quantization error, which inevitably limits their performance. To overcome these limitations, another set of techniques has emerged that follows the point-based pipeline, which operates directly on raw point clouds to learn point-wise features [15, 31, 34, 39, 45, 49, 51]. DGCNN [39] and PointNet++ [32] are two popular point-based backbones, and the promising

results of these methods make them widely utilized backbones [34, 45, 49]. Current state-of-the-art point cloud accelerators also focus on accelerating them [7, 8, 19, 41].

Since points in a point cloud are arbitrarily spread, point cloud algorithms require explicit neighbor search and point aggregation operations. Generally, a layer of point cloud neural networks can be abstracted as the following processes [7, 8, 32, 39, 41]. Each input point undergoes a neighbor search process based on the distance of points' geometric positions or features. Subsequently, neighbor message transfer is performed based on the neighbor information, including feature update and neighbor feature information aggregation.

In the initial setup, every point is identified by its unique set of (x, y, z) coordinates, which represent the point's geometric position in space. Points that are geometrically adjacent to each other tend to have similar (x, y, z) coordinates. The features of points are then generated by multiple layers which include neighbor search and neighborhood message passing. As the features of each point in multiple layers are generated based on its original coordinates and those of its neighbors' coordinates. Despite the feature generation process, similarities between geometrically adjacent points may still persist due to their proximity in space. Moreover, existing accelerators [8, 19] for point cloud neural networks show that explicit neighbor search, feature update, and feature aggregation all affect the execution latency of the point cloud neural network. *Our motivation lies in investigating the existence of execution redundancies within geometrically adjacent of points in all the execution processes of point cloud neural networks. By reducing these redundancies, we aim to improve the speed of point cloud neural networks.*

In this study, we first conducted a comprehensive analysis of the execution flow of general point cloud neural networks. Our analysis uncovers strong similarities and correlations between geometrically adjacent points in three key aspects. (i) The neighbors of points tend to be located in close proximity to each other in space. (ii) Geometrically adjacent points often share a significant number of common neighbors. (iii) Geometrically adjacent points tend to have similar features, despite the feature generation process. These multiple geometric similarities provide a great opportunity to improve the execution efficiency of point cloud networks. Although similarity exploration has been studied for convolutional neural networks (CNN) in the field of architecture [24, 33], these efforts have mainly focused on exploiting feature similarity and cannot be applied to multiple geometric similarities present in point clouds. Meanwhile, existing point cloud algorithms and accelerators [7, 8, 19, 41, 47] do not leverage these multiple geometric similarities to accelerate point cloud neural networks. This may be because point cloud neural networks involve multiple complex operations (e.g., explicit neighbor searching, feature updating, and aggregation), making it a non-trivial task to leverage this multi-dimensional similarity.

To address these observations and challenges, we design a systematic software hardware co-design approach to end to end explore geometric similarity to accelerate the execution of point cloud neural networks. We first propose a novel geometry-aware differential point cloud neural network to address the observed multiple similarities. The network consists of three modules: geometry-aware neighbor search, geometry-aware differential update, and geometry-aware differential aggregation. These modules utilize

both geometry-aware and differential execution model to optimize the neighbor search process, and accelerate neighbor search, feature update and aggregation respectively by reducing computation and communication redundancies.

Although the redundancies can be reduced by the proposed algorithm, it is not supported well by current accelerators [7, 8, 19, 41, 47]. To tackle this, we propose a geometry-aware differential point cloud accelerator (GDPCA) with which the redundancy reductions can be turned into performance improvement. Specifically, GDPCA has three core engines: *Geometry-aware Neighbor & Aggregation Group Generate Engine*, *Differential Update Engine*, and *Differential Aggregate Engine*. These three engines work in a pipeline to support the execution flow of our proposed algorithm. GDPCA follows a geometry-aware bulk execution model, during which the points are grouped in geometric adjacent bulks and are executed one by one. The points in one group exhibit strong similarities, which makes the designed engines can leverage these similarities to reduce computation and communication redundancies.

To the best of our knowledge, our proposed GDPCA is the first systematic software hardware co-design approach to end-to-end explore geometric similarity in point cloud neural networks. To summarize, we have made the following contributions:

- We first reveal that geometrically adjacent points exhibit strong similarities and correlations in point cloud neural networks.
- We propose a novel algorithm that utilizes these geometrically based similarities and correlations to reduce computation and communication redundancies.
- We further propose a novel architecture design to help efficiently identify and reduce redundant computations and communications on-the-fly, transforming the redundancy reduction into performance improvement.
- Compared with Crescent [7], the state-of-the-art point cloud accelerator, GDPCA gains an average of $2.5\times$ speedup and $2.6\times$ energy-efficiency.

2 BACKGROUND AND ANALYSIS

2.1 Point Cloud Networks and Abstraction

Point cloud usually comprises a set of unordered points $X = (x_1, x_2, \dots, x_n)$, where each point $x_i \in \mathbb{R}^F$ contains F -dimensional features. Initially, x_i is uniquely identified by its (x, y, z) coordinates. Researchers designed unique point-based neural networks, including PointNet++ [32] and DGCNN [39], which extract the local features for each point by aggregating information from neighboring points (including self-loops). As illustrated in Fig. 1, they follow the consistent computation paradigm, and the whole computation process can be abstracted into four main phases [7, 8, 47].

Neighbor Search. Since points in the point cloud are arbitrarily distributed in 3D space, the point cloud network requires an explicit neighbor search to return the K nearest neighbors of point x_i . Common neighbor search methods include K -nearest neighbor search (KNN), K -d tree construction, and ball query [27, 32]. Taking KNN as an example, given the input point cloud X , the neighbors

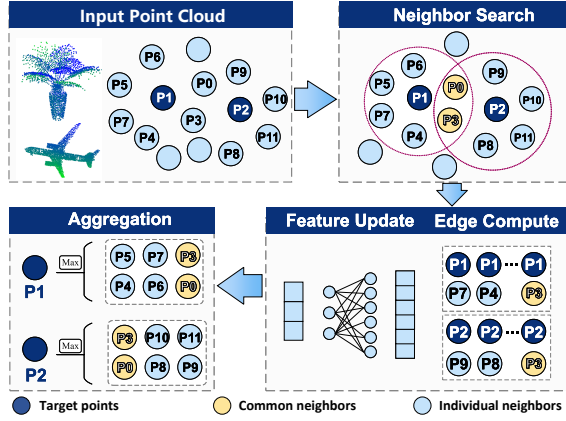


Figure 1: Illustration of a layer of point cloud neural networks.

of the point x_i are computed by the Euclidean distance:

$$j_k = \arg \min_{x_j \in X} D(x_i, x_j) = \arg \min_{x_j \in X} \|x_i - x_j\|^2, \quad (1)$$

where $j_k \in \mathbb{R}^K$ denotes the k -th neighbor index of point x_i .

Edge Compute. After obtaining the neighborhood information of each point, the point cloud network performs the edge compute operation to obtain the edge features of the point. Specifically, given the points x_i and its neighbors $x_{j_k} \in N(x_i)$, we utilize an edge function $h(\cdot)$ to compute the local point-to-point relationship for each neighbor:

$$e_{ij_k} = h(x_{j_k}, x_i) = x_{j_k} - x_i, \quad (2)$$

where e_{ij_k} denotes the edge feature of the edge (x_{j_k}, x_i) .

Feature Update. It leverages a multi-layer perceptron (MLP) to perform feature extraction to update the edge features of each neighbor, i.e.,

$$x_{ij_k} = \sigma(w \cdot e_{ij_k}), \quad (3)$$

where x_{ij_k} represents the updated edge feature, w is the learnable parameters, and σ is an activation function, e.g. Relu.

Aggregate. Finally, the aggregate phase employs an aggregation function to gather feature messages from the neighbors to obtain the output features. For example, given the updated edge features of point x_i , we can utilize max-pooling as the aggregation function to yield the output features $\hat{x}_i$:

$$\hat{x}_i = \max_{1 \leq k \leq K} x_{ij_k}. \quad (4)$$

Delayed Aggregation. To reduce computation redundancy and increase feature computation reuse, some work [8, 47] proposed the delayed aggregation to re-customize the computation processes of point cloud networks as shown below:

$$\hat{x}_i = \max_{1 \leq k \leq K} \sigma(w \cdot (x_{j_k} - x_i)) \approx \sigma \left(\max_{1 \leq k \leq K} (w \cdot x_{j_k}) - w \cdot x_i \right). \quad (5)$$

The central idea is to delay the edge compute until after the feature update, that is, it combines the edge compute with the original

aggregate operation to form a new delayed aggregation operation. The benefits come from two aspects: (i) The neighbor search phase and the feature update phase can be executed in parallel, which improves the execution performance. (ii) It allows the network to perform feature updates directly on raw input features instead of relying on normalized edge features $(x_{j_k} - x_i)$. It effectively reduces the computation of feature updates from the number of edges to the number of points.

2.2 Performance Analysis

We conduct a performance analysis of the point cloud network to understand the bottleneck of delayed aggregation. Fig. 2 shows the latency breakdown on the ModelNet40 and KITTI datasets for the two most representative point cloud networks (Pointnet++ and DGCNN). We can see that neighbor search and delay aggregation are major bottlenecks for point cloud network acceleration, cumulatively accounting for more than 80% of total latency (DGCNN up to 90% in KITTI datasets). Despite the slight difference in the execution pipeline of delayed aggregation, this observation is similar to the latest point cloud accelerator PointAcc [19]. Meanwhile, feature aggregation accounts for 17% of the total latency on average. Thus, explicit neighbor search, feature update, and feature aggregation all affect the execution latency of point cloud neural networks, which implies that they all require acceleration.

2.3 Analysis of Geometric Similarity in Point Cloud Neural Networks

In point cloud neural networks, inputs are the unordered (x, y, z) coordinates of points, representing their geometric positions. Similar coordinate values can indicate adjacent points. The neural networks use these initial values to generate point features. It encourages us to study the geometric similarity in point cloud neural networks.

Geometric Neighbor Distribution. Neighbor search constructs the neighborhood information for each point by the distance metric. Due to the **geometric locality** of the point cloud, we conjecture that the neighbors of a specific point may exist in the geometrically adjacent regions of this point. To verify the above conjecture, we first voxelize the point cloud. The data structure can be defined as follows.

Definition 1 (Voxelization data structure): Given the input point cloud X , we voxelize it by setting up a voxel size (v_x, v_y, v_z) , which divides the whole point cloud into 3D regular grids. Each point is mapped to its corresponding voxel index as:

$$V_{(u,v,w)} = \left(\left\lfloor \frac{x}{v_x} \right\rfloor, \left\lfloor \frac{y}{v_y} \right\rfloor, \left\lfloor \frac{z}{v_z} \right\rfloor \right), \quad (6)$$

where u, v, w indicates the grid that point would fall into.

Based on Definition 1, we define H -hop voxel to describe the neighbor distribution. Specifically, if the neighbor of a point is distributed among neighboring voxels with distance H , this neighbor is termed as H -hop voxel. Fig. 3 illustrates the neighbor cumulative distribution on ModelNet40 [43] and KITTI [12] datasets. The distribution is measured over the two most common models, PointNet++ and DGCNN. We can easily observe that in KITTI dataset, the neighbors of points are concentrated in the range of 2-hop voxel, and in ModelNet40 dataset, the neighbors of points are distributed within

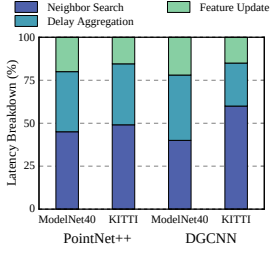


Figure 2: Latency Breakdown across the four main point cloud operations. The data is averaged on A100 GPU over 100 executions.

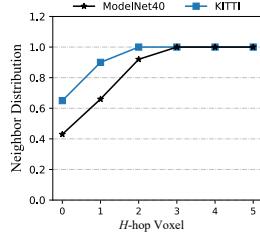


Figure 3: Neighbor distribution on KITTI and ModelNet40 datasets.

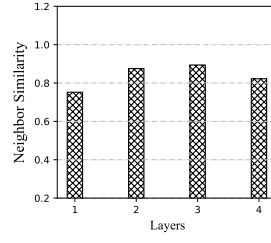


Figure 4: Average neighbor similarity of DGCNN on KITTI dataset at different neural network layers.

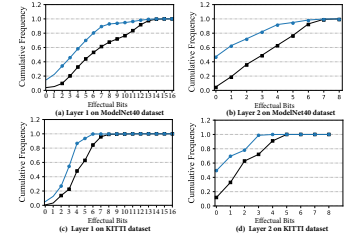


Figure 5: Cumulative distribution of the number of effectual bits per raw feature/delta over KITTI and ModelNet40 dataset under different quantization schemes.

the range of 3-hop voxel. Nevertheless, current point cloud neural networks algorithms and accelerators [7, 8, 19, 32, 39, 41] adopt the brute-force search approach, which performs the neighbor search by calculating the distance for all points. It introduces large-scale unnecessary computation and communication.

Geometric Neighbor Similarity. Based on the above observation, we further hypothesize that geometrically close points contain extensive common neighbors. Given a voxel grid V_i with s points, we define the neighbor similarity N_s of V_i as the proportion of the common neighbor set of all points in V_i to the neighbors, i.e.,

$$N_s = \frac{|\bigcap_{x_i \in V_i} N(x_i)| \cdot s}{\sum_{x_i \in V_i} |N(x_i)|}, \quad (7)$$

where $\bigcap_{x_i \in V_i} N(x_i)$ is the common neighbor set of V_i .

Fig. 4 statistics the average neighbor similarity (the average of neighbor similarities across all voxel grids) of DGCNN on KITTI dataset at different neural network layers. We can easily find that the neighbor similarity exceeds 75.2% for all layers, and even reaches 89.4% at layer 3. It effectively indicates that geometrically adjacent points contain many overlapping neighbors. This observation provides a precious opportunity to reuse the intermediate results of aggregation to improve performance. However, existing point cloud networks [8, 39] employ the neighborhood message-passing mechanism to process each point independently. This naive implementation would result in the inability to identify and utilize the intermediate computational results of these common neighbors, thus incurring massive computational redundancy.

Geometric Feature Similarity. By analyzing the execution flow of point cloud neural networks, we conjecture that features of geometrically close points have high similarity. Here, we adopt the delta approach [24] to measure feature similarity. In particular, given a non-empty voxel $V_{(u,v,w)}$, we compute the grid center p_c of $V_{(u,v,w)}$ based on the position, and the base point x_c is defined as the point nearest to $V_{(u,v,w)}$. The remaining points in $V_{(u,v,w)}$ except the base point x_c can be expressed in a differential form:

$$x_i = x_c + (x_i - x_c) = x_c + \Delta_i, \quad (8)$$

where Δ_i is the element-wise deltas among x_i and x_c . If the original features exhibit strong spatial correlations, Δ_i is small. This

provides an opportunity to reduce the required computational workload, as the delta Δ_i requires only fewer effectual bits to represent compared to the original features. Let's first define effectual bits: the multiplication $x_i \times w$ of node x_i and weight w . If x_i is represented by m bits, the multiplication is equivalent to adding p terms, where the m -th term is the result of multiplying the m -th bit of the multiplier x_i with the shifted m bit positions multiplicand w :

$$x_i \times w = \sum_{m=0}^{m=p} x_{i_m} \cdot (w \ll m). \quad (9)$$

Only the bits of x_i that are set to 1 contribute to effectual work. These effectual bits represent the number of bits in x_i that have a value of 1. Fig. 5 illustrates the cumulative distribution of the number of effectual bits for both original features and delta features across different neural network layers. The measurements were conducted on the KITTI dataset and ModelNet40 dataset using DGCNN with different quantization schemes. The right column presents the distributions of effectual bits with 8-bit quantization, while the left column displays the distributions with 16-bit quantization. Taking Fig. 5(b) as an example, it demonstrates that nearly 1% of the original features have 0 effectual bits, whereas 49.4% of the delta features have 0 effectual bits. This implies that the sparsity of layer 2 on the KITTI dataset, when using delta representations, reaches 49.4%. Moreover, it can be observed that the sparsity of the delta (the fraction of zero values) rapidly increases as the number of layers increases. These figures show that the delta representation contains significantly fewer effectual bits for feature values, indicating a significant potential for reducing the number of required computations if delta features are processed instead of original features.

3 ALGORITHM DESIGN OF GDPCA

GDPCA is a systematic software hardware co-design approach, which aims to exploit the geometric similarities of point clouds to accelerate point cloud networks. In this section, we first discuss the algorithm design of GDPCA.

3.1 Definitions

We first introduce some definitions to better understand our proposed algorithm.

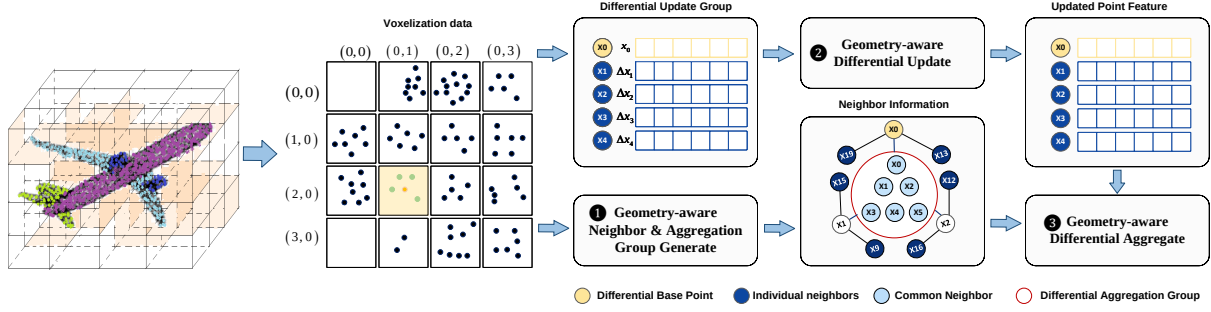


Figure 6: Illustration of geometry-aware differential point cloud neural networks (GDPCA). GDPCA contains three main sub-procedures: geometry-aware neighbor & aggregation group generation, geometry-aware differential update, and geometry-aware differential aggregate.

Definition 2 (Differential Update Group): Given a non-empty voxel $V_{(u,v,w)}$, we define all points in the $V_{(u,v,w)}$ as a differential update group, i.e., $UG_{V_i} : \{x_i \in V_{(u,v,w)}\}$.

Definition 3 (Differential Base Point): Given a differential update group UG_{V_i} , we compute the grid center p_c of UG_{V_i} based on the position, and the differential base point x_c can be defined as the point nearest to p_c in UG_{V_i} .

Definition 4 (Differential Aggregation Group): Given a differential update group UG_{V_i} , the differential aggregation group AG_{V_i} can be defined as a common neighbor set for all points in UG_{V_i} , i.e., $AG_{V_i} : \{\bigcap_{x_i \in UG_{V_i}} N(x_i)\}$.

3.2 Algorithm Overview

As shown in Fig. 6, given an input point cloud X , we first voxelize it into a 3D regular grid, and map each point to its corresponding voxel index to generate differential update groups $UG_{V_i} \in X$. GDPCA contains three core phases. ① **Geometry-aware Neighbor & Aggregation Group Generate**, which generate differential aggregation groups on-the-fly while realizing efficient neighbor search. ② **Geometry-aware Differential Update**, which exploits the feature similarity of the point cloud to update the feature representation in delta execution manner for reducing the computation. ③ **Geometry-aware Differential Aggregate**, which performs differential aggregation by reusing the intermediate aggregation results of common neighbors to eliminate redundant aggregation. The details are described as follows.

3.3 Geometry-aware Neighbor & Aggregation Group Generate

Based on the analysis in Section 2.3, the neighbors of points are distributed in geometrically close regions. To exploit this property, we propose a novel geometry-aware neighbor search algorithm to achieve efficient neighbor search by reducing the region of neighbor search. Specifically, the algorithm searches only in geometrically close H -hop voxels, rather than in the whole data space. Given the differential update group UG_{V_i} as input, UG_{V_i} and its H -hop neighboring voxels as the search region R_{x_i} . For each target point $x_i \in UG_{V_i}$, traverses all points in R_{x_i} , calculate their distance from x_i , and perform topk operation to search the K neighbors with the smallest distance for point x_i . In this manner, it is feasible to

improve the efficiency of neighbor search without losing accuracy. Note that the proposed algorithm is scalable, and any advanced neighbor search can be more efficient by exploring the geometric neighbor distributions.

In addition, the points in UG_{V_i} share many common neighbors due to the neighbor similarity. To facilitate subsequent aggregate operations to reuse the intermediate computation results of common neighbors, we innovatively combine the generation of differential aggregation groups with neighbor search. That is, while searching neighbors for UG_{V_i} , the differential aggregation group is dynamically generated online. A more detailed discussion can be found in Section 4.2.

3.4 Geometry-aware Differential Update

Due to the strong local geometric correlations of point clouds, the geometrically close points have a high feature similarity. To this end, we design a novel geometry-aware differential update operation, which utilizes the deltas to perform feature updates for reducing computation. Specifically, given a differential update group UG_{V_i} , we first utilize the raw features to perform feature updates on the differential base points x_{c_i} , while the remaining points $x_{V_i} \in UG_{V_i}$ can be computed in an incremental manner according to Equ. 8:

$$\begin{aligned} \bar{x}_c &= w \cdot x_c, \\ \bar{x}_{V_i} &= w \cdot x_{V_i} = w \cdot (x_c + \Delta_{V_i}) = w \cdot x_c + w \cdot \Delta_{V_i}. \end{aligned} \quad (10)$$

Fig. 7 presents an example of the original feature update operation and the LDU operation. The given UG_{V_i} contains 4 points, where x_0 is the difference base point. The original feature update operation directly utilizes the shared weight to extract features from the four points. In contrast, LDU only utilizes the raw features for feature update at x_0 , while the remaining points (x_1, x_2, x_3) are computed with deltas. Finally, the result of the base point (x_0) is added to the other incremental results to reconstruct the correct output. In this manner, we utilize the deltas with fewer effectual bits instead of the original feature, which greatly reduces the effort of feature updating.

3.5 Geometry-aware Differential Aggregate

The aggregate phase generates the final output by aggregating the feature messages from the neighbors. Based on the analysis

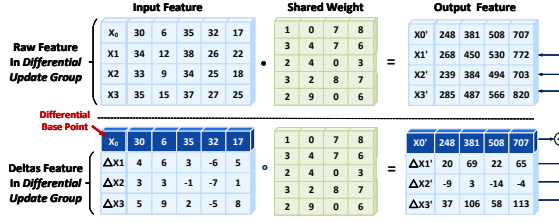


Figure 7: Illustration of geometry-aware differential update.

of Section 2.3, geometrically close points contain extensive common neighbors. Original aggregation operations aggregate each point independently, and the overlapping common neighbors are recomputed each time, resulting in extensive redundant aggregation. Taking Fig. 8(a) as the example, given three target points to be updated (x_0, x_1, x_2), each including 8 neighbors (including the self-loop). These target points share a common set of neighbors ($x_0, x_1, x_2, x_3, x_4, x_5$), i.e., the differential aggregation group AG_{V_i} . The original aggregate operation requires repeating the aggregation of common neighbors 3 times.

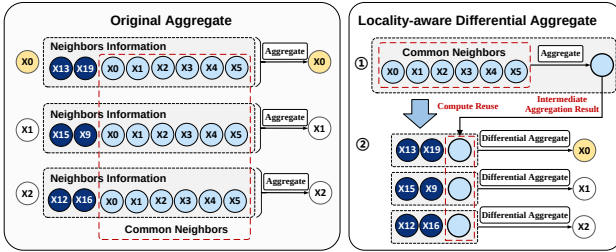


Figure 8: (a) Original aggregation operations aggregate each point independently, and the overlapping common neighbors are recomputed each time. (b) Geometry-aware differential aggregation aggregates the common neighbors and caches the intermediate aggregate results for compute reuse, and then treats the individual neighbors of the target node as increments for differential aggregation operations.

To address this issue, we propose a geometry-aware differential aggregate algorithm. As shown in Fig. 8(b), we first aggregate the common neighbors in the differential aggregate group AG_{V_i} , and cache the intermediate aggregate results $\bar{x}_{com}$ for reuse. Subsequently, differential aggregate is performed for each target point by reusing the intermediate aggregation results $\bar{x}_{com}$. In this case, the individual neighbors of the target node are treated as increments. Formally, given a node x_{V_i} to be aggregated and its neighborhood information $N(V_i)$, the computational flow of LDA can be expressed as follows:

$$\begin{aligned} \bar{x}_{com} &= \max_{j \in AG_{V_i}} (\bar{x}_j), \\ x_{V_i} &= \max \left(\bar{x}_{com}, \max_{m \in (N(V_i) - AG_{V_i})} (\bar{x}_m) \right), \end{aligned} \quad (11)$$

where $\bar{x}_{com}$ is the intermediate aggregation results of the differential aggregate group AG_{V_i} .

3.6 The Need for a Customized Accelerator

Existing general-purpose architectures (e.g., GPUs) may not release the potential of differential point cloud algorithms well for two reasons: (i) GPU is inherently optimized for computationally intensive workloads with regular execution pattern [20], such as neural networks. In contrast, the proposed differential point cloud algorithms contain both regular feature update operations, and irregular neighbor search and aggregation operations, resulting in lower computational utilization and efficiency of the GPU. (ii) Bit-serial based incremental processing involves processing data from bit-by-bit, and GPU cannot support such operations well.

Existing point cloud accelerators [8, 19, 47] also cannot well support the proposed differential point cloud algorithm. The reasons are as follows: (i) Existing accelerators can not support the efficient online generation of differential aggregation groups. A straightforward implementation is to first transfer the features of all points from off-chip memory to on-chip memory, perform the neighbor search on these vertices, and store the neighborhood information in the off-chip memory. The host program (e.g., CPU) is then used to find the differential aggregation group. This approach introduces additional computation cycles and involves large-scale unnecessary communication overhead. (ii) The differential update operation based on bit-serial require existing accelerators to redesign the feature computing processing unit. (iii) Existing accelerators do not support differential aggregation operations. The reason is that existing accelerators cannot identify which nodes belong to differential aggregation groups and common neighbor groups, making it impossible to cache intermediate results by direct memorization.

4 ARCHITECTURE DESIGN OF GDPCA

4.1 Architecture Overview

Design Overview. GDPCA is a pipelined architecture accelerator that integrates multiple sub-computing engines, as shown in Fig. 9. It consists of three core execution engines: *Geometry-aware Neighbor & Aggregation Group Generate Engine*, *Differential Update Engine*, and *Differential Aggregate Engine*. Different hardware computing engines are designed for different computing parts and optimized separately to improve intra-block and inter-block parallelism. This fine-grained hybrid architecture can effectively increase data reuse and reduce unnecessary irregular access to alleviate sparsity. GDPCA follows a geometry-aware bulk execution model, during which the points are grouped in geometric adjacent bulks and are executed one by one. The points in one group exhibit strong similarities, which makes the designed engines can leverage these similarities to reduce computation and communication redundancies.

The *Geometry-aware Neighbor & Aggregation Group Generate Engine* innovatively combines neighbor search with online generation of differential aggregation groups. It first improves the efficiency of the neighbor search process by reducing the scope of the neighbor search, reducing both execution operations and on-chip data access. We also cleverly use the results of the neighbor search to dynamically generate differential aggregation groups, reducing the need for additional clock cycles and communication overhead.

The *Differential Update Engine* is designed to support the regular feature update operations for each bulk. It also supports differential delta updates, which significantly reduces the computation of feature update by using the feature similarity of the point cloud.

The *Differential Aggregation Engine* ensures that point features in the differential aggregation group AG_{v_i} are loaded only once and computed only once for aggregation, and then the intermediate results can be reused for target points in UG_{V_i} . This greatly reduces the computation overhead and communication overhead caused by redundant aggregation. Additionally, we implement a vertex-disperse processing method to exploit the point parallelism and intra-point parallelism.

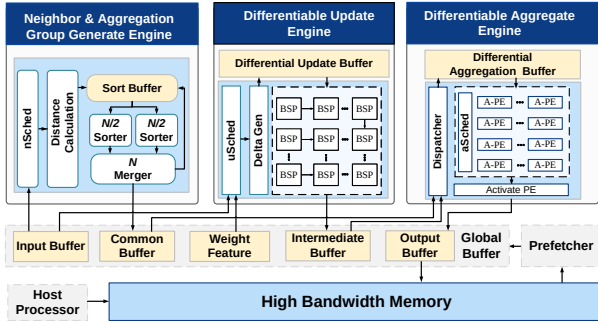


Figure 9: Architecture overview of GDPCA.

Buffer Management. To reduce data access latency, we employ on-chip buffers (*Global Buffers*) to cache various data and intermediate results to improve data reuse. *Input Buffer* and *Weight Buffer* are utilized to store the input feature and weight parameters, *Common Buffer* is used to store the indexes of neighbors, and AG_{v_i} generated by *Geometry-aware Neighbor & Aggregation Group Generate Engine*, and *Intermediate Buffer* is utilized to store the intermediate results generated by *Differential Update Engine*. To hide the access latency, almost all buffers adopt double-buffering technology [3, 4] to allow overlapping execution of different operations.

Execution Flow of GDPCA. ❶ The *Host Processor* first voxelizes the input point cloud to generate differential update groups and determines the neighbor search range for each differential update group. The pre-processed results are then stored in the High Bandwidth Memory (HBM). ❷ A *Prefetcher* is used to explicitly prefetch the point cloud features and weight parameters from HBM group-by-group, and cache them into the *Input Buffer* and *Weight Buffer*. ❸ *Geometry-aware Neighbor & Aggregation Group Generate Engine* fetches the neighbor search workload from *input buffer* to perform the neighbor search and cache the results to the *Common Buffer*. ❹ *Differential Update Engine* fetches the feature update workload from *Input Buffer* and *Weight Buffer* to perform feature extraction and cache the results into the *Intermediate Buffer*. ❺ Finally, *Differential Aggregation Engine* aggregates neighbor information based on the results of the previous two phases to generate the output features, and writes them to the *Output Buffer*. Note that, the *Geometry-aware Neighbor & Aggregation Group Generate Engine* and *Differential Update Engine* are executed in parallel, which greatly improves pipeline parallelism and increases execution efficiency.

4.2 Design of Geometry-aware Neighbor & Aggregation Group Generate Engine

In this section, we first introduce an online generation scheme to simultaneously support the efficient neighbor search and differential aggregation group AG_{V_i} generation.

Online Generation of Differential Aggregation Group Scheme.

A naive implementation is to traverse the points in UG_{V_i} , and then search neighbors for each point independently. After all the points are processed, the indexes of the common neighbors are generated by the merge search operation, i.e., AG_{V_i} . However, this simple method would introduce numerous additional merge search operations. Meanwhile, the merge search operation needs to be executed after all points' neighbors have been computed, which introduces additional computation cycles.

To tackle these issues, we design an online generation scheme to generate the AG_{V_i} on-the-fly in an iterative updating manner. Fig. 10(a) illustrates the specific execution procedure. Given the input point cloud UG_{V_i} , ❶ we first perform the geometry-aware neighbor search on the differential base point x_{c_i} , and initialize AG_{V_i} with the search results i.e., the neighbor index of x_{c_i} . ❷ Then, we select a point $x_i \in \{UG_{V_i} - x_{c_i}\}$ as the next processing point. In this case, the search region R_{x_i} of x_i can be divided into two sets: the current AG_{V_i} and $\{R_{x_i} - AG_{V_i}\}$. ❸ The distances of all points in AG_{V_i} to x_i are first calculated, and sorted them in ascending order based on their distances to generate the *neighbor array*. Subsequently, we perform the similar operation on the set $\{R_{x_i} - AG_{V_i}\}$ to x_i , and utilize the topk operation to select k points to generate the *candidate neighbor array*. ❹ We utilize the interpolation-merging operation to merge two arrays. Specifically, we replace the larger distance points in the *neighbor array* with the smaller distance points in the *candidate neighbor array* to generate the neighbors of x_i . Meanwhile, the points that are not replaced in the *neighbor array* are the updated AG_{V_i} . Since the two arrays are sorted, the interpolation-merging operation only requires $O(1)$ time complexity. ❺ Traverse the remaining points in UG_{V_i} , and repeat steps ❷–❹ until all points in UG_{V_i} have been processed to generate the final AG_{V_i} .

Through this online generation mechanism, we reuse the results of the neighbor search cleverly, significantly reducing the complexity of the merge search without introducing additional clock cycles.

Architecture of Geometry-aware Neighbor & Aggregation Group Generate Engine. As shown in Fig. 10(b), it contains a task scheduler (*nSched*), a *distance calculator*, two MUX, two *N/2 sorters*, a *N merger*, and a *Sort Buffer*. The *nSched* is designed to fetch the workload of the neighbor search from the *Input Buffer*. The *distance calculator* is used to compute the distance, and the results are cached in the *Sort Buffer*. To accommodate the online generation Scheme, we divide the *Sort Buffer* into *neighbor slot* and *candidate neighbor slot*, which store *neighbor array* and *candidate neighbor array* respectively. Two *N/2 sorters* and *N merger* are used to perform sorting and topk operation. The *N/2 sorter* read $n/2$ distance values from the *Sort Buffer* at a time for ascending sort, and the *N merger* merge-sort two sorted arrays into one array. The

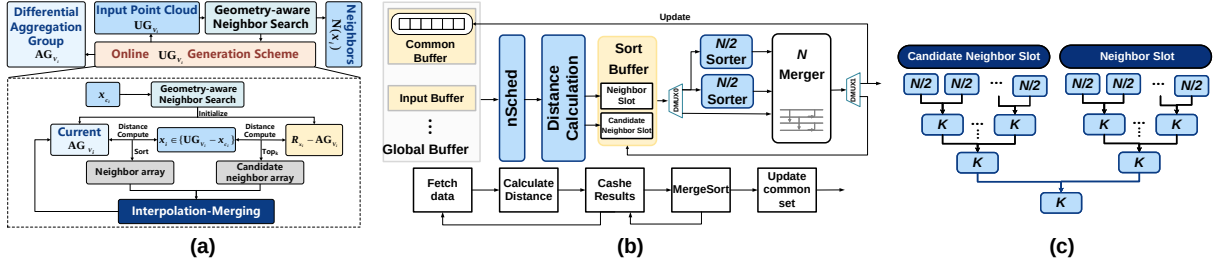


Figure 10: (a) Illustration of online generation of differential aggregation group AG_{V_i} . (b) Architecture design of the Geometry-aware Neighbor & Aggregation Group Generate Engine. (c) Workflow of the Geometry-aware Neighbor & Aggregation Group Generate Engine.

$topk$ operation is supported by truncating the sorted array to k length.

Configurable Execution Pipeline. For a UG_{V_i} , ❶ we first randomly initialize the differential aggregation group AG_{V_i} in *Common Buffer*. ❷ Using *nsched* to read the workload for the target points x_i in UG_{V_i} . It is divided into two sub-stages: i) Read the points in AG_{V_i} , perform distance calculation, and cache them to the *neighbor slot* in *Sort Buffer*; ii) Read and compute all points in R_{x_i} except AG_{V_i} , and cache them to the *candidate neighbor slot*. ❸ As shown in Fig. 10(c), two $N/2$ Sorters and N Merger are used to perform sort and $topk$ operations on the *neighbor slot* and *candidate neighbor slot* respectively. ❹ Since N Merger can only handle fixed-length arrays, we utilize DMUX1 to determine whether all elements of both slots have been processed (0:false, 1: true). If DMUX1 outputs 0, we use the *Sort Buffer* to store the intermediate result and repeat step ❸ until all elements have been executed. ❺ After the *neighbor slot* and *candidate neighbor slots* are processed, DMUX0 is used to control the dataflow to perform the interpolation-merging operation. If DMUX0 outputs 1, the two sorted arrays are fed to generate the neighbor index of x_i and update the AG_{V_i} . ❻ Traverse the remaining points in UG_{V_i} , and repeat steps ❷–❺ until all nodes have been processed.

4.3 Design of Differential Update Engine

To support the proposed geometry-aware differential update algorithm, we design a novel *Differential Update Engine*. As shown in Fig. 9, it consists of a *uSched*, a *delta generator*, a *Differential Update Buffer*, and a *Differential Update Processing Unit (DU-PU)*. The *uSched* aims to fetch the updated workload from the *Input Buffer* and *Weight Buffer*. The *delta generator* calculates the deltas by subtracting element-by-element and caches the deltas in the *Differential Update Buffer*. The DU-PU is the core element of the Differential Update Engine and is responsible for performing delta processing. It builds upon the classic Bit-Pragmatic accelerator (PRA) [1], which processes the input features bit-serially and only processes the effectual bits of the input features. Therefore, the execution time of the PRA is proportional to the number of effectual bits of the features. Due to the feature similarity of the point cloud, the effectual bits of the deltas are much smaller than the raw features. We modify the basic architecture of PRA to support delta processing to translate this phenomenon into performance improvements.

Fig. 11(a) shows the architecture of DU-PU. Similar to PRA [1], DU-PU contains $P_n \times P_c$ ($P_n = 16, P_c = 16$) *offset generators*, and $P_n \times P_c$ basic bit-serial processing element (BSP). Each BSP column deals with a point to be updated, and $BSP(i, j)$ deals with i -dimension features of the j -th point. In each cycle, we load the P_c dimensional features of P_n points to be updated, and $P_n \times P_c$ *offset generators* run in parallel and convert the features into streams with an effective power of 2 through the modified booth encoding. The offset and weight are then broadcast to the corresponding BSP for update processing. Fig. 11(b) shows the internals of the BSP. Each BSP has a P_c input adder tree, and P_c shifters (not multipliers). Each offset controls a shifter, effectively multiplying the weight by a power of 2. The shifted weights are reduced by the adder tree. To further support processing deltas, each BSP is configured with a differential reconstruction processing element (DRP), which is responsible for reconstructing the final output after delta processing. It uses MUX to allow BSPs to calculate output features using deltas or raw feature values. The *output reg* is used to cache the output features.

Execution Pipeline for Differential Update Engine. We first utilize *uSched* to fetch the features of the first $P_n = 16$ points in UG_{V_i} including the differential base points x_{c_i} . The *delta generator* is then used to convert the original feature into deltas, and stores them to the *Differential Update Buffer*. x_{c_i} is expressed by the original feature and the other points in UG_{V_i} are expressed in deltas. Next, the *offset generator* is utilized to convert the feature in *Differential Update Buffer* into a stream with an effective power of 2 and broadcast to the corresponding BSP. The BSPs in column 0 are assigned to handle x_{c_i} with raw features, while the BSPs in columns 1-15 are assigned to process the other 15 points with increments. When the BSPs of column 0 complete the computation, the result is written to *output reg* and broadcast to the corresponding DRPs in the other 15 columns. Columns 1-15 employ the DRPs to update their differential output to normal feature values. Finally, the updated features are written to the *Intermediate Buffer*.

Supports Arbitrary Scale Processing. Since DU-PU can only handle $P_n = 16$ points at a time, which is far away from the scale of the point feature. To support handling arbitrary scale of UG_{V_i} , we further introduce a *state reg* and a MUX to control the execution dataflow. The *state reg* records whether the x_{c_i} has been processed (0: false, 1: true). If the *state reg* is 0, x_{c_i} has not been processed, and we process x_{c_i} normally with the BSP in column 0 and write the result back to the *Differential Update Buffer* for reuse. At this time,

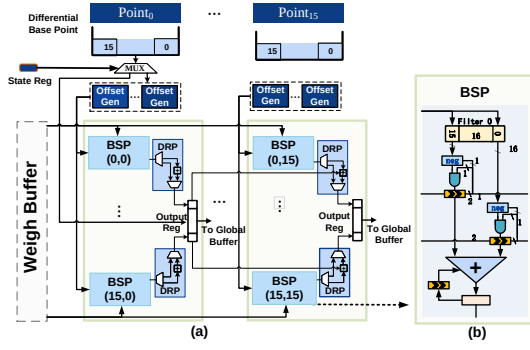


Figure 11: Architecture design of a *Differential Update Processing Unit (DU-PU)*.

we change the state of *state reg* to 1. If the *state reg* is 1, we skip the calculation of column 0 and utilize MUX to load the updated feature of x_{c_i} into *output reg* in column 0 for subsequent differential reconstruction.

4.4 Design of Differential Aggregate Engine

The *Differential Aggregate Engine* is designed to support our proposed differential aggregation algorithm.

Architecture of Differential Aggregate Engine. Fig. 12 shows the architecture of the *Differential Aggregate Engine*, which contains a *Dispatcher*, a *Differential Aggregator Buffer* and an *Differential Aggregation Processing Unit (DA-PU)*. The *Dispatcher* aims to dispatch aggregation workload from *Common Buffer* and *Intermediate Buffer*. The *Differential Aggregator Buffer* is used to store a workload of the aggregate task. It is composed of multiple *task slot*. Each *task slot* stores the workload of a point to be aggregated. The *task slot* contains a *feature buffer* to store the neighbor features, and each neighbor feature corresponds to a *task type reg* that distinguishes whether the neighbor is in the differential aggregation group AG_{v_i} (0: false, 1: true), and a *state reg* to denote whether this point is the differential base point x_{c_i} (0: false, 1: true).

DA-PU fetches the aggregation tasks in the *Differential Aggregator Buffer* and executes accordingly. It contains an aggregation scheduler *aSched*, an aggregation processing element (A-PE) group, a *Base Reg*, and two MUXs for controlling the dataflow. The *aSched* ensures the pipeline execution and assigns the workload to A-PE group. A-PE group is responsible for Max aggregate operation. MUX1 is used to cache the intermediate results of AG_{v_i} , while MUX2 is utilized to reuse the intermediate results.

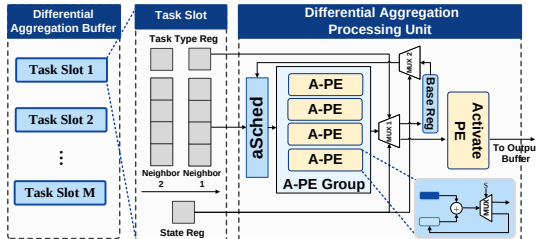


Figure 12: Architecture design of a *Differential Aggregate Unit*.

Execution Workflow. The execution workflow of the *Differential Aggregate Engine* consists of two distinct phases. (i) **Aggregation workload preparation:** the *Dispatcher* feeds the aggregated workload into *Differential Aggregator Buffer* according to the results of the first two stages. The workload of the x_{c_i} is stored in *task slot 0* (*state reg* is 1), and its neighbors are divided into two components: AG_{v_i} (*task type reg* is 1) and individual neighbors (*task type reg* is 0). while the remaining *task slots* store only the corresponding individual neighbors by truncating AG_{v_i} . (ii) **Aggregation phase,** *aSched* polls the *task type reg* and *state reg* and forwards them to MUX1 and MUX2. For x_{c_i} (*state reg* is 1), if the neighbor in AG_{v_i} (*task type reg* is 1), the intermediate result is cached after the A-PE group completes the workload; Otherwise, no saving is required. For the other points in UG_{v_i} (*state reg* is 0), the intermediate results are reused directly, and all individual neighbors are computed sequentially until processing is completed (*task type reg* is 0). Finally, *Activate PEs* perform the activation operation to produce the output feature.

In this manner, we can reuse the intermediate results of differential aggregation groups AG_{v_i} in bulk on-the-fly. This effectively eliminates redundant aggregations and saves a large amount of computation and on-chip data access.

5 EVALUATION RESULTS

5.1 Experimental Setup

Methodology. The performance and energy of GDPCA are measured using the following tools. We implement a validated custom cycle-accurate simulator to get the cycle numbers. The simulator models the microarchitectural behavior of each module, which is integrated with Ramulator [17] to estimate HBM timings and generate a command trace. For power and area measurements of each module, we implement each module in Verilog, and utilize the Synopsys Design Compiler (DC) with the TSMC 40 nm standard VT library for the synthesis, and estimate the power consumption using Synopsys PrimeTime PX. The area and power of on-chip memory are calculated by CACTI [37]. We adopt the quantization-aware training with PyTorch [16] for all the evaluated baselines models to reduce the precision to 16b in our experiment, which introduces negligible accuracy loss compared with 32b.

Table 1: Evaluated models and datasets.

Dataset	Algorithm	Application Domains
ModelNet40 [43]	PointNet++(c) [32]	Classification
	DGCNN(c) [39]	
	LDGCNN [48]	
	DensePoint [21]	
KITTI [12]	PointNet++(s) [32]	Segmentation
	DGCNN(s) [39]	
	F-PointNet [30]	

Evaluated Models and Compared Baselines. To evaluate GDPCA, we utilize five different point cloud network benchmarks based on two popular datasets as shown in Table 1. In the evaluation, we compare GDPCA with two kinds of hardware baselines. The first

category is the server-level product. We compare the full-size implementation GDPCA against the CPU platform with Intel Xeon(R) CPU E5-2680 v3 CPUs and 500GB DRAM, and a GPU platform equipped with the NVIDIA Tesla A100. The second category is the state-of-the-art point cloud accelerators, including Crescent [7], PointAcc [19], and Mesorasi [8]. To enable a fair comparison, we enhance the configuration of the baseline accelerators as GDPCA. For Mesorasi, we equip it with a neighbor search engine same as Crescent. More detailed configurations are listed in Table 2.

Table 2: System configurations of compared accelerators.

Chip	Mesorasi	PointAcc	Crescent	GDPCA(ours)
Cores	64 × 64 Systolic MAC Unit Array	64 × 64 Systolic MAC Unit Array	64 × 64 Systolic MAC Unit Array	64 × 64 BSP Array & 128 A-PEs
SRAM(KB)	2048	776	2048	2560 (1728 KB (Global), 64 KB (Sort), 384 KB (Aggregate) 384 KB (Update))
Area (mm ²)	14.2	15.7	13.6	16.5
Frequency	1GHz	1GHz	1GHz	1GHz
DRAM	HBM 2	HBM 2	HBM 2	HBM 2
Bandwidth	256 GB/s	256 GB/s	256 GB/s	256 GB/s
Technology	40 nm	40 nm	40 nm	40 nm

5.2 Overall Results

Accuracy. We focus on two factors that affect accuracy: the hop factor in our geometry-aware neighbor search and delayed aggregation. In Fig. 13(a), we investigate the impact of delayed aggregation on the original networks. The results demonstrate that, despite the approximation operation involved in delayed aggregation, accuracy is not compromised. Additionally, Fig. 13(b), illustrates the evaluation of GDPCA with different hop factors. Notably, we select 2 (for KITTI) and 3 (for ModelNet40) as the hop factors (each object in ModelNet40 is partitioned into $8 \times 8 \times 8$ voxels and the size of each voxel in KITTI is $0.3m \times 0.3m \times 0.3m$), since exceeding these values does not degrade accuracy. This is because most neighboring points exist within these hop ranges, thus capturing essential local information. Note that these hop factors are dataset-specific, the selection of an appropriate hop factor should be based on the characteristics of the dataset, such as point density and distribution.

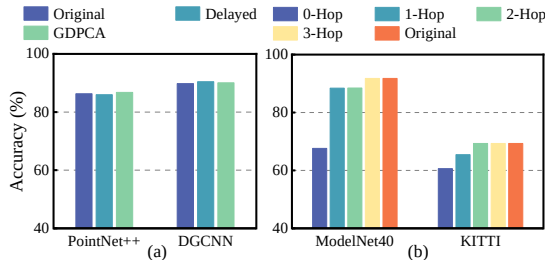


Figure 13: Effect of accuracy determinants. (a) Delayed aggregation's effect. (b) Different hop factors.

Speedup. In this experiment, we adopt the total number of execution cycles as the performance metric. Fig. 14 depicts the performance of our method compared with other baselines, including

Mesorasi, PointAcc, Crescent, CPU, and GPU. On average, GDPCA is 4.4×, 1.8×, 2.5×, 161.1× and 4.2× faster than Mesorasi, PointAcc, Crescent, CPU, and GPU respectively. Such improvements can be attributed to several key features. In the neighbor search stage, the proposed *Geometry-aware Neighbor & Aggregation Group Generate Engine* searches only in geometrically close regions, rather than the entire data space, reducing the search complexity and the number of required computations. In the update phase, our *Differential Update Engine* capitalizes on geometric similarity by processing only the deltas (differences between features) rather than the raw features. This approach leverages the inherent feature similarity among points within proximate regions, resulting in a reduced computational demand. In the aggregation stage, our *Differential Aggregate Engine* avoids redundant aggregations and DRAM accesses for elements in the redundant aggregation set by leveraging the neighbor similarity and reusing the intermediate aggregation result within a voxel.

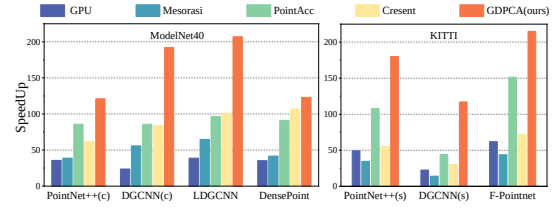


Figure 14: Comparison of speedup over CPU.

DRAM Access. In this experiment, we evaluate the effectiveness of our method in reducing DRAM accesses compared to the baseline accelerators. On average, GDPCA achieves 83%, 66%, and 58% DRAM reduction compared with Mesorasi, PointAcc, and Crescent, respectively. Our method significantly decreases the number of DRAM accesses, which can be attributed to the proposed *Geometry-aware Neighbor & Aggregation Group Generate Engine*. Due to the limited buffer space, the compared accelerators need to repeatedly load the entire point features for KNN calculation, causing substantial DRAM overhead. In contrast, our *Geometry-aware Neighbor & Aggregation Group Generate Engine* effectively exploits the geometric similarity to minimize the number of loaded neighbors for searching. Additionally, it reuses the intermediate aggregation result of redundant aggregation sets, substituting multiple DRAM accesses to individual point features with a single access to the aggregated feature of the respective aggregation group.

Energy Consumption. Fig. 15 presents the normalized energy efficiency comparison between our method and other baseline accelerators. Overall, GDPCA achieves energy savings of 2.6×, 1.5×, 4.2× compared to Crescent, PointAcc, and Mesorasi, respectively. Our method achieves energy savings by reducing redundant computations through geometry-aware techniques and optimizing DRAM energy efficiency by leveraging point cloud data's inherent geometry. Fig. 16 illustrates the energy breakdown of our method across the two most typical benchmarks, PointNet++ and DGCNN. On average, the DRAM, *Geometry-aware Neighbor & Aggregation Group Generate Engine*, *Differential Update Engine*, *Differential Aggregate Engine*, the on-chip buffer, and the others account for 22.8%, 31.8%,

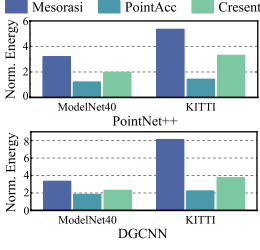


Figure 15: Energy savings of GDPCA over baselines.

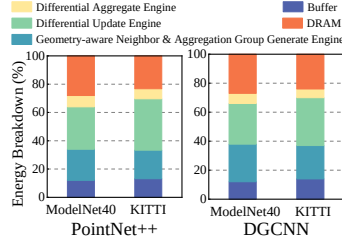


Figure 16: Energy breakdown of GDPCA on typical models.

7.0%, 12.8% and 25.6% of the total energy consumption, respectively. Over 60% of the total energy is consumed by the computations engines. Additionally, the energy control overhead such as generating the differential aggregation group on the fly and the offsets for the deltas is minimal in comparison to the saved energy.

Area Overhead Breakdown. The total area of GDPCA is 16.5 mm^2 . The area percentages for the components are shown in Table 3. Most of the area is designed for computation engines. In comparison to the baseline accelerators, GDPCA has a larger area. The additional hardware area is utilized to effectively generate the differential aggregation group online and to generate offsets for efficiently calculating the deltas. This additional hardware significantly reduces computational and memory access overhead. Furthermore, GDPCA contains larger on-chip buffers, which serve to cache more intermediate results of aggregation groups, substantially enhancing execution efficiency.

Table 3: Area breakdown of GDPCA.

Module	Area (%)
Geometry-aware Neighbor & Aggregation Group Generate Engine	28.3
Differential Update Engine	43.1
Differential Aggregate Engine	13.2
Total Buffer	12.8
Others	2.6

5.3 Effects of Different Geometry-aware Scheme

Effects of Geometry-aware Neighbor Search. We demonstrate the effectiveness of the proposed geometry-aware neighbor search by comparing the performance of GDPCA with GDPCA_KNN that represents the variant of GDPCA that searches for KNN neighbors in the entire point cloud space in Fig. 17(a). The result demonstrates that GDPCA can significantly reduce the computation time and memory access. In particular, the performance improvement is more pronounced for the KITTI dataset, which is a real-world dataset containing a large number of points in each point cloud and exhibits enhanced spatial locality compared to ModelNet40. Our search method can find complete neighbors using fewer hops, highlighting the increased effectiveness of our method for accelerating the processing of large-scale point clouds in real-world applications while maintaining high accuracy levels.

Effects of Geometry-aware Differential Update. We compare the performance of GDPCA with a variant (termed as GDPCA_SYS) that employs a traditional systolic array, which is adopted by all the compared baseline accelerators in their update stage instead of the *Differential Update Engine*. To ensure a fair comparison, the systolic array is configured to have the same hardware configuration as our DUE. Shown in Fig. 17(b), our experimental results demonstrate that the architecture with the DUE consistently achieves significant speedup in execution time compared to the one using the traditional systolic array. This improvement can be attributed to the utilization of deltas in the update operation, which reduces the number of effectual bits to be processed than the raw values. For instance, with our 16-bit quantization configuration, the mean count of effectual bits for a 4-layer DGCNN is [13, 7, 7, 7] for ModelNet40, and [8, 5, 5, 5] for KITTI.

Furthermore, our results reveal that the speedup obtained by GDPCA is higher for KITTI. This is because the strong local geometric correlations in KITTI result in a higher feature similarity among neighboring points, leading to fewer effectual bits and less computation in the delta computation of these points.

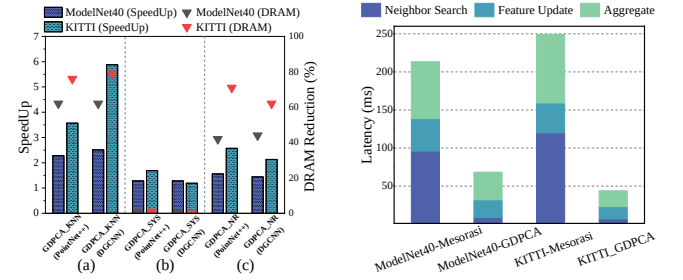


Figure 17: Effect of Geometry-aware Schemes.

Figure 18: Latency breakdown for both Mesorasi and our GDPCA.

Effects of Geometry-aware Differential Aggregate. To validate the effectiveness of our proposed Geometry-aware differential aggregate method, we designed the GDPCA_NR variant, which does not utilize the intermediate results of the differential aggregate group (AG) for comparison. In Fig. 17(c), the experimental results demonstrate that our method's speedup outperforms the non-reuse scenario of GDPCA_NR. The advantage of our LDA method comes from its ability to exploit the neighbor similarity and reuse intermediate results within the AG, which helps to reduce the computation and DRAM access for aggregation.

Latency Breakdown. Fig. 18 presents the latency breakdown of PointNet++ for both Mesorasi and our GDPCA on benchmark datasets. Mesorasi's latency is primarily influenced by the neighbor search stage due to high computation and DRAM access demands, particularly for the KITTI dataset with a large number of points. Our geometry-aware methods significantly reduce latency across all stages, with the most substantial reduction in the neighbor search stage by avoiding unnecessary processing of non-local points. However, the geometric-aware update stage sees a less noticeable reduction as it only eliminates computations on non-effectual bits, without saving any DRAM access.

6 RELATED WORK

Deep Learning on Point Clouds. Deep learning-based point cloud analytics has received extensive attention in a range of emerging intelligent systems, such as robotics [22, 26, 42], AR/VR devices [28, 36], and autonomous vehicles [6, 46]. PointNet [31], proposed a point-based approach for point clouds. Motivated by PointNet, a series of point-based variants have been proposed, including, PointNet++ [32], DGCNN [39], LDGCNN [48], and DensePoint [21], etc. In addition, advanced FlatFormer [23] has begun to explore the transformer for 3D representation learning, and achieved promising performance. Similar to point-based methods, It also utilizes initial coordinates to extract local features of points, thus they both have the geometric similarity of point clouds. This provides an opportunity to improve the execution efficiency of transformer-based methods.

Point Cloud Accelerators. As the development of deep learning on point cloud neural networks, some accelerators [7, 8, 19, 41, 47] which aim at accelerating point cloud neural networks have been proposed. Mesorasi [8] introduced an accelerator built through delayed-aggregation to support PointNet++ and DGCNN. PointAcc [19] solved the sparsity issue of point clouds by caching on-demand streaming sparse calculations and temporarily fusing consecutive dense computational layers, and can be compatible with different point cloud networks. Point-X [47] exploited spatial locality to improve communication efficiency in the processing of point clouds with graph-based DNN, primarily for DGCNN. Feng et al. [7] proposed Crescent, an architecture of taming memory Irregularities for accelerating point cloud neural networks. Nevertheless, these point cloud accelerators ignore the enormous accelerating potential brought about by the intrinsic geometric correlation of the point cloud.

Accelerator for exploring locality and similarity. Some studies [25, 47] focused on utilizing locality to address cache, communication, etc. Other work [24, 33] exploited feature similarity to accelerate the execution of CNN. Diffy [24] exploited the spatial correlation to introduce hardware accelerators that perform differential convolution, reducing the computation and communication required. DeltaRNN [10], EdgeDRNN [11], and Spartus [9] leveraged the temporal similarity in RNN for incremental computation. However, the multiple geometric similarities of point clouds and the complex execution pipeline of point cloud networks make these accelerators unsuitable.

7 CONCLUSION

In this work, we reveal that geometrically adjacent points exhibit strong similarities and correlations in point cloud neural networks. A naive implementation of point cloud neural networks leads to redundant computations and communications. Enlightened by this, we propose a geometry-aware differential point cloud accelerator, shortly termed as GDPCA, which can exploit geometric similarity to reduce these redundancies. GDPCA is supported by an algorithm and architecture co-design. We first propose a novel algorithm that can discover and reduce computation and communication redundancies by our proposed geometry-aware and differential execution mechanisms. Then we propose a novel architecture design to help

identify and reduce redundant computations and communications, transforming the execution reduction into performance improvement through hardware support. To the best of our knowledge, GDPCA is the first accelerator that can reduce execution redundancies for point cloud neural networks by exploiting geometric similarity.

ACKNOWLEDGMENTS

This work was partially supported by the Science Fund for Creative Research Groups of the National Natural Science Foundation of China under Grant 62321003, and by the Fundamental Research Funds for the Central Universities under Grant x2wjD2230230.

REFERENCES

- [1] Jorge Albericio, Alberto Delmás, Patrick Judd, Sayeh Sharify, Gerard O'Leary, Roman Genov, and Andreas Moshovos. 2017. Bit-pragmatic deep neural network computing. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*. 382–394.
- [2] Waleed Ali, Sherif Abdelkarim, Mahmoud Zidan, Mohamed Zahran, and Ahmad El Sallab. 2018. Yolo3d: End-to-end real-time 3d oriented object bounding box detection from lidar point cloud. In *Proceedings of the European conference on computer vision (ECCV) workshops*. 0–0.
- [3] Cen Chen, Kenli Li, Yangfan Li, and Xiaofeng Zou. 2022. ReGNN: A redundancy-eliminated graph neural networks accelerator. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 429–443.
- [4] Cen Chen, Kenli Li, Xiaofeng Zou, and Yangfan Li. 2021. Dygnn: Algorithm and architecture support of dynamic pruning for graph neural networks. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1201–1206.
- [5] Siheng Chen, Baoan Liu, Chen Feng, Carlos Vallespi-Gonzalez, and Carl Welling-ton. 2020. 3d point cloud processing and learning for autonomous driving: Impacting map creation, localization, and perception. *IEEE Signal Processing Magazine* 38, 1 (2020), 68–86.
- [6] Yaodong Cui, Ren Chen, Wenbo Chu, Long Chen, Daxin Tian, Ying Li, and Dongpu Cao. 2021. Deep learning for image and point cloud fusion in autonomous driving: A review. *IEEE Transactions on Intelligent Transportation Systems* (2021).
- [7] Yu Feng, Gunnar Hammonds, Yiming Gan, and Yuhao Zhu. 2022. Crescent: taming memory irregularities for accelerating deep point cloud analytics. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. 962–977.
- [8] Yu Feng, Boyuan Tian, Tiancheng Xu, Paul Whatmough, and Yuhao Zhu. 2020. Mesorasi: Architecture support for point cloud analytics via delayed-aggregation. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1037–1050.
- [9] Chang Gao, Tobi Delbruck, and Shih-Chii Liu. 2022. Spartus: A 9.4 TOP/s FPGA-based LSTM accelerator exploiting spatio-temporal sparsity. *IEEE Transactions on Neural Networks and Learning Systems* (2022).
- [10] Chang Gao, Daniel Neil, Enea Ceolini, Shih-Chii Liu, and Tobi Delbruck. 2018. DeltaRNN: A power-efficient recurrent neural network accelerator. In *Proceedings of the 2018 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. 21–30.
- [11] Chang Gao, Antonio Rios-Navarro, Xi Chen, Shih-Chii Liu, and Tobi Delbruck. 2020. EdgeDRNN: Recurrent neural network accelerator for edge inference. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 10, 4 (2020), 419–432.
- [12] Andreas Geiger, Philip Lenz, and Raquel Urtasun. 2012. Are we ready for autonomous driving? the kitti vision benchmark suite. In *2012 IEEE conference on computer vision and pattern recognition*. IEEE, 3354–3361.
- [13] Yulan Guo, Mohammed Bennamoun, Ferdous Sohel, Min Lu, and Jianwei Wan. 2014. 3D object recognition in cluttered scenes with local surface features: A survey. *IEEE transactions on pattern analysis and machine intelligence* 36, 11 (2014), 2270–2287.
- [14] Yulan Guo, Ferdous Sohel, Mohammed Bennamoun, Min Lu, and Jianwei Wan. 2013. Rotational projection statistics for 3D local surface description and object recognition. *International journal of computer vision* 105, 1 (2013), 63–86.
- [15] Yulan Guo, Hanyun Wang, Qingyong Hu, Hao Liu, Li Liu, and Mohammed Bennamoun. 2020. Deep learning for 3d point clouds: A survey. *IEEE transactions on pattern analysis and machine intelligence* (2020).
- [16] Sagar Imambi, Kolla Bhanu Prakash, and GR Kanagachidambaresan. 2021. Py-Torch. *Programming with TensorFlow: Solution for Edge Computing Applications* (2021), 87–104.
- [17] Yoongu Kim, Weikun Yang, and Onur Mutlu. 2015. Ramulator: A fast and extensible DRAM simulator. *IEEE Computer architecture letters* 15, 1 (2015), 45–49.

- [18] Alex H Lang, Sourabh Vora, Holger Caesar, Lubing Zhou, Jiong Yang, and Oscar Beijbom. 2019. Pointpillars: Fast encoders for object detection from point clouds. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 12697–12705.
- [19] Yujun Lin, Zhekai Zhang, Haotian Tang, Hanrui Wang, and Song Han. 2021. Pointacc: Efficient point cloud accelerator. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 449–461.
- [20] Erik Lindholm, John Nickolls, Stuart Oberman, and John Montrym. 2008. NVIDIA Tesla: A unified graphics and computing architecture. *IEEE micro* 28, 2 (2008), 39–55.
- [21] Yongcheng Liu, Bin Fan, Gaofeng Meng, Jiwen Lu, Shiming Xiang, and Chunhong Pan. 2019. Denspoint: Learning densely contextual representation for efficient point cloud processing. In *Proceedings of the IEEE/CVF international conference on computer vision*. 5239–5248.
- [22] Zhijian Liu, Alexander Amini, Sibozhu, Sertac Karaman, Song Han, and Daniela Rus. 2021. Efficient and Robust LiDAR-Based End-to-End Navigation. *arXiv e-prints* (2021), arXiv–2105.
- [23] Zhijian Liu, Xinyu Yang, Haotian Tang, Shang Yang, and Song Han. 2023. FlatFormer: Flattened Window Attention for Efficient Point Cloud Transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 1200–1211.
- [24] Mostafa Mahmoud, Kevin Siu, and Andreas Moshovos. 2018. Diffy: A Déjà vu-free differential deep neural network accelerator. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 134–147.
- [25] Anurag Mukkara, Nathan Beckmann, Maleen Abeydeera, Xiaosong Ma, and Daniel Sanchez. 2018. Exploiting locality in graph analytics through hardware-accelerated traversal scheduling. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1–14.
- [26] Ayrton Oliver, Steven Kang, Burkhard C Wünsche, and Bruce MacDonald. 2012. Using the Kinect as a navigation sensor for mobile robotics. In *Proceedings of the 27th conference on image and vision computing New Zealand*. 509–514.
- [27] Chungyun Park, Yoonwoo Jeong, Minsu Cho, and Jaesik Park. 2022. Fast point transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 16949–16958.
- [28] Alessio Pierluigi Placitelli and Luigi Gallo. 2011. Low-cost augmented reality systems via 3D point cloud sensors. In *2011 Seventh International Conference on Signal Image Technology & Internet-Based Systems*. IEEE, 188–192.
- [29] François Pomerleau, Francis Colas, Roland Siegwart, et al. 2015. A review of point cloud registration algorithms for mobile robotics. *Foundations and Trends® in Robotics* 4, 1 (2015), 1–104.
- [30] Charles R Qi, Wei Liu, Chenxia Wu, Hao Su, and Leonidas J Guibas. 2018. Frustum pointnets for 3d object detection from rgb-d data. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 918–927.
- [31] Charles R Qi, Hao Su, Kaichun Mo, and Leonidas J Guibas. 2017. Pointnet: Deep learning on point sets for 3d classification and segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 652–660.
- [32] Charles Ruizhongtai Qi, Li Yi, Hao Su, and Leonidas J Guibas. 2017. Pointnet++: Deep hierarchical feature learning on point sets in a metric space. *Advances in neural information processing systems* 30 (2017).
- [33] Marc Riera, Jose-Maria Arnau, and Antonio González. 2018. Computation reuse in DNNs by exploiting input similarity. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 57–68.
- [34] Weijing Shi and Raj Rajkumar. 2020. Point-gnn: Graph neural network for 3d object detection in a point cloud. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 1711–1719.
- [35] Martin Simon, Karl Amende, Andrea Kraus, Jens Honer, Timo Samann, Hauke Kaulbersch, Stefan Milz, and Horst Michael Gross. 2019. Complexer-yolo: Real-time 3d object detection and tracking on semantic point clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*. 0–0.
- [36] Jonathan Dyssel Stets, Yongbin Sun, Wiley Corning, and Scott W Greenwald. 2017. Visualization and labeling of point clouds in virtual reality. In *SIGGRAPH Asia 2017 Posters*. 1–2.
- [37] S. Thoziyoor, N. Muralimanohar, and N. P. Jouppi. [n. d.]. CACTI 5.0. *Date* ([n. d.]).
- [38] Peng-Shuai Wang, Yang Liu, Yu-Xiao Guo, Chun-Yu Sun, and Xin Tong. 2017. O-cnn: Octree-based convolutional neural networks for 3d shape analysis. *ACM Transactions On Graphics (TOG)* 36, 4 (2017), 1–11.
- [39] Yue Wang, Yongbin Sun, Ziwei Liu, Sanjay E Sarma, Michael M Bronstein, and Justin M Solomon. 2019. Dynamic graph cnn for learning on point clouds. *Acm Transactions On Graphics (tog)* 38, 5 (2019), 1–12.
- [40] Yue Wang, Shusheng Zhang, Bile Wan, Weiping He, and Xiaoliang Bai. 2018. Point cloud and visual feature-based tracking method for an augmented reality-aided mechanical assembly system. *The International Journal of Advanced Manufacturing Technology* 99, 9 (2018), 2341–2352.
- [41] Zilun Wang, Wendong Mao, Peixiang Yang, Zhongfeng Wang, and Jun Lin. 2022. An Efficient FPGA Accelerator for Point Cloud. In *2022 IEEE 35th International System-on-Chip Conference (SOCC)*. IEEE, 1–6.
- [42] Mark Whitty, Stephen Cossell, Kim Son Dang, Jose Guivant, and Jayantha Katupitiya. 2010. Autonomous navigation using a real-time 3d point cloud. In *2010 Australasian Conference on Robotics and Automation*. 1–3.
- [43] Zhirong Wu, Shuran Song, Aditya Khosla, Fisher Yu, Linguang Zhang, Xiaoou Tang, and Jianxiong Xiao. 2015. 3d shapenets: A deep representation for volumetric shapes. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 1912–1920.
- [44] Bin Yang, Wenjie Luo, and Raquel Urtasun. 2018. Pixor: Real-time 3d object detection from point clouds. In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*. 7652–7660.
- [45] Zetong Yang, Yanan Sun, Shu Liu, and Jiaya Jia. 2020. 3dssd: Point-based 3d single stage object detector. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 11040–11048.
- [46] Xiangyu Yue, Bichen Wu, Sanjit A Seshia, Kurt Keutzer, and Alberto L Sangiovanni-Vincentelli. 2018. A lidar point cloud generator: from a virtual world to autonomous driving. In *Proceedings of the 2018 ACM on International Conference on Multimedia Retrieval*. 458–464.
- [47] Jie-Fang Zhang and Zhengya Zhang. 2021. Point-X: A spatial-locality-aware architecture for energy-efficient graph-based point-cloud deep learning. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 1078–1090.
- [48] Kuang Zhang, Ming Hao, Jing Wang, Clarence W de Silva, and Chenglong Fu. 2019. Linked dynamic graph cnn: Learning on point cloud via linking hierarchical features. *arXiv preprint arXiv:1904.10014* (2019).
- [49] Yifan Zhang, Qingyong Hu, Guoquan Xu, Yanxin Ma, Jianwei Wan, and Yulan Guo. 2022. Not all points are equal: Learning highly efficient point-based detectors for 3d lidar point clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 18953–18962.
- [50] Yin Zhou and Oncel Tuzel. 2018. Voxelnet: End-to-end learning for point cloud based 3d object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 4490–4499.
- [51] Xiaofeng Zou, Kenli Li, and Cen Chen. 2020. Multilevel attention based u-shape graph neural network for point clouds learning. *IEEE Transactions on Industrial Informatics* 18, 1 (2020), 448–456.